

# OrderPulse — Step 3: Inventory Microservice Core

Technical Instruction & Execution Blueprint for AI Developer Agents

## OBJECTIVE

Build the Inventory Service using Uber Go guidelines (uber-go/fx for DI and uber-go/zap for structured logging). Implement the gRPC server backed by PostgreSQL and Redis.

## Step 3 Tasks

- **1. Directory Structure:** Setup Uber/Clean Architecture package layout under `cmd/inventory/` and `internal/inventory/`.
- **2. Dependency Injection & Logging:** Initialize `zap.Logger` and wire application dependencies using `uber-go/fx`.
- **3. Repositories:** Create PostgreSQL store for stock state and Redis cache for stock lookups.
- **4. gRPC Handler:** Implement `CheckStock` (Redis cached) and `ReserveStock` (Postgres transactional).
- **5. Server Lifecycle:** Hook gRPC server lifecycle into `fx.Lifecycle` on port `:50051`.

## Directory Layout Template

```
orderpulse/
├── cmd/
│   └── inventory/
│       └── main.go
├── internal/
│   └── inventory/
│       ├── config/
│       │   └── config.go
│       ├── domain/
│       │   └── item.go
│       ├── repository/
│       │   ├── postgres.go
│       │   └── redis.go
│       ├── service/
│       │   └── inventory.go
│       ├── grpc/
│       │   └── handler.go
│       └── module.go
├── proto/
│   └── inventory/
│       └── v1/
```

cmd/inventory/main.go

```
package main

import (
    "go.uber.org/fx"
    "orderpulse/internal/inventory"
)

func main() {
    fx.New(
        inventory.Module,
    ).Run()
}
```

**internal/inventory/module.go**

```

package inventory

import (
    "context"
    "net"

    "go.uber.org/fx"
    "go.uber.org/zap"
    "google.golang.org/grpc"

    "orderpulse/internal/inventory/config"
    inventorygrpc "orderpulse/internal/inventory/grpc"
    "orderpulse/internal/inventory/repository"
    inventorysvc "orderpulse/internal/inventory/service"
    inventoryv1 "orderpulse/proto/inventory/v1"
)

var Module = fx.Options(
    fx.Provide(
        config.Load,
        zap.NewProduction,
        repository.NewPostgresRepository,
        repository.NewRedisRepository,
        inventorysvc.NewInventoryService,
        inventorygrpc.NewHandler,
    ),
    fx.Invoke(RegisterGRPCServer),
)

func RegisterGRPCServer(
    lc fx.Lifecycle,
    logger *zap.Logger,
    cfg *config.Config,
    handler *inventorygrpc.Handler,
) {
    server := grpc.NewServer()
    inventoryv1.RegisterInventoryServiceServer(server, handler)

    lc.Append(fx.Hook{
        OnStart: func(ctx context.Context) error {
            lis, err := net.Listen("tcp", cfg.GRPCPort)
            if err != nil {
                return err
            }
            logger.Info("Starting Inventory gRPC Server", zap.String("port", cfg.GRPCPort))
            go server.Serve(lis)
            return nil
        },
        OnStop: func(ctx context.Context) error {
            logger.Info("Stopping Inventory gRPC Server")
            server.GracefulStop()
            return nil
        },
    })
}

```

```
}))  
}
```

## Verification Commands

```
# Install Uber Fx and Zap dependencies  
go get go.uber.org/fx go.uber.org/zap  
  
# Run Inventory Service  
go run cmd/inventory/main.go  
  
# Query gRPC Endpoint using grpcurl  
grpcurl -plaintext -d '{"product_id": "prod-101", "quantity": 2}' localhost:50051 inventory.v1.Inver
```